Customer Service Request Analysis

Task 1: Understand the Dataset

1.1 Import Dataset

```
import pandas as pd
df = pd.read_csv("311_Service_Requests_from_2010_to_Present.csv")
```

```
C:\Users\test\AppData\Local\Temp\ipykernel_3504\310309575.py:2: DtypeWarning: Columns (48,49) have mixed types. Specify dtype op
  df = pd.read_csv("311_Service_Requests_from_2010_to_Present.csv")
```

1.2 View Data

```
df.head()
```

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Location Type	Incident Zip	Incident Address	...	Bridg Highwa Nam
0	32310363	12/31/2015 11:59:45 PM	01/01/2016 12:55:15 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party	Street/Sidewalk	10034.0	71 VERMILYEA AVENUE	...	Na
1	32309934	12/31/2015 11:59:44 PM	01/01/2016 01:26:57 AM	NYPD	New York City Police Department	Blocked Driveway	No Access	Street/Sidewalk	11105.0	27-07 23 AVENUE	...	Na
2	32309159	12/31/2015 11:59:29 PM	01/01/2016 04:51:03 AM	NYPD	New York City Police Department	Blocked Driveway	No Access	Street/Sidewalk	10458.0	2897 VALENTINE AVENUE	...	Na
3	32305098	12/31/2015 11:57:46 PM	01/01/2016 07:43:13 AM	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking	Street/Sidewalk	10461.0	2940 BAISLEY AVENUE	...	Na
4	32306529	12/31/2015 11:56:58 PM	01/01/2016 03:24:42 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	Street/Sidewalk	11373.0	87-14 57 ROAD	...	Na

5 rows × 53 columns

1.3 Shape

```
df.shape
```

```
(364558, 53)
```

1.4 Columns

```
df.columns
```

```
Index(['Unique Key', 'Created Date', 'Closed Date', 'Agency', 'Agency Name',
       'Complaint Type', 'Descriptor', 'Location Type', 'Incident Zip',
       'Incident Address', 'Street Name', 'Cross Street 1', 'Cross Street 2',
       'Intersection Street 1', 'Intersection Street 2', 'Address Type',
       'City', 'Landmark', 'Facility Type', 'Status', 'Due Date',
       'Resolution Description', 'Resolution Action Updated Date',
       'Community Board', 'Borough', 'X Coordinate (State Plane)',
       'Y Coordinate (State Plane)', 'Park Facility Name', 'Park Borough',
       'School Name', 'School Number', 'School Region', 'School Code',
```

```
       'School Phone Number', 'School Address', 'School City', 'School State',
       'School Zip', 'School Not Found', 'School or Citywide Complaint',
       'Vehicle Type', 'Taxi Company Borough', 'Taxi Pick Up Location',
       'Bridge Highway Name', 'Bridge Highway Direction', 'Road Ramp',
       'Bridge Highway Segment', 'Garage Lot Name', 'Ferry Direction',
       'Ferry Terminal Name', 'Latitude', 'Longitude', 'Location'],
      dtype='object')
```

1.5 Null Values

```
print(df.isnull().sum())
```

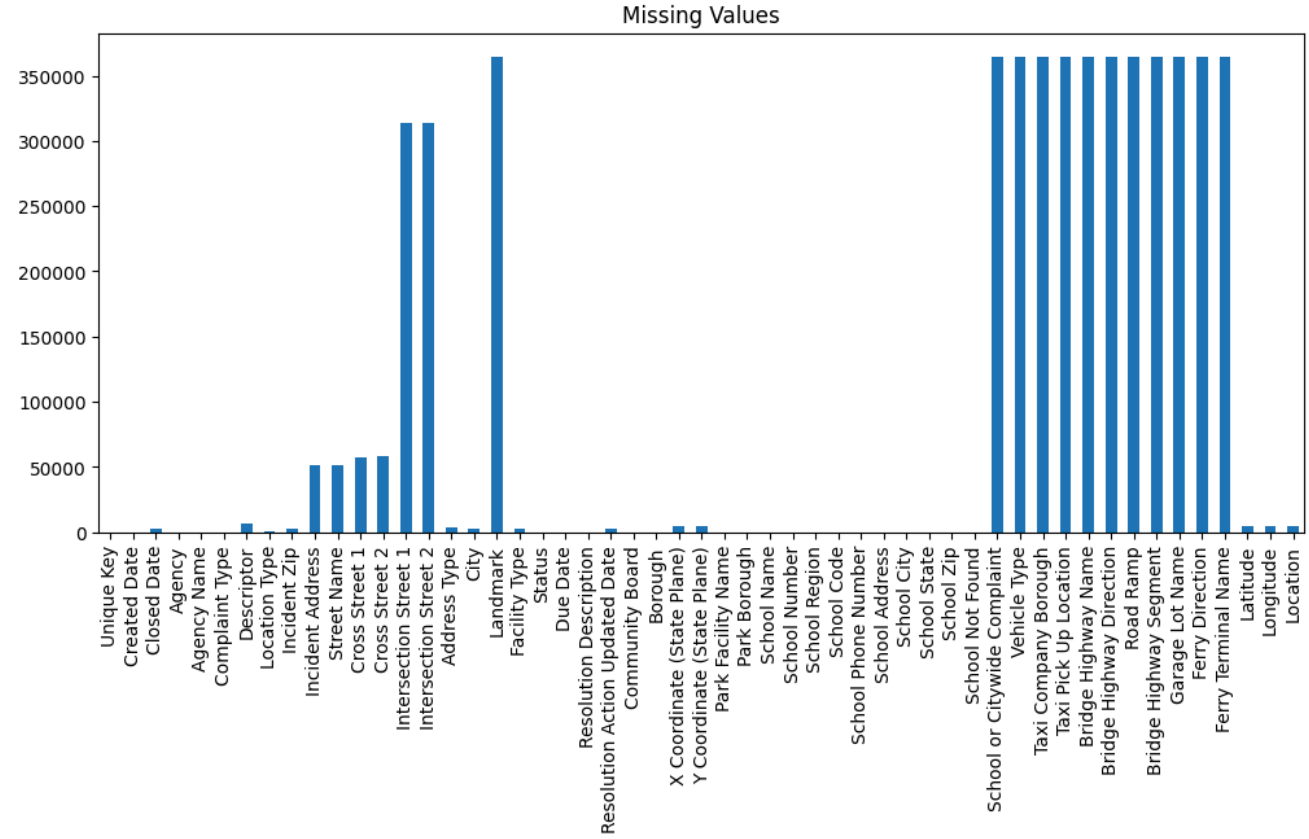
```
Unique Key                           0
Created Date                         0
Closed Date                       2381
Agency                               0
Agency Name                          0
Complaint Type                       0
Descriptor                        6501
Location Type                      133
Incident Zip                      2998
Incident Address                 51699
Street Name                      51699
Cross Street 1                   57188
Cross Street 2                   57805
Intersection Street 1           313438
Intersection Street 2           314046
Address Type                      3252
City                              2997
Landmark                        364183
Facility Type                     2389
Status                               0
Due Date                             3
Resolution Description               0
Resolution Action Updated Date    2402
Community Board                      0
Borough                              0
X Coordinate (State Plane)        4030
Y Coordinate (State Plane)        4030
Park Facility Name                   0
Park Borough                         0
School Name                          0
School Number                        0
School Region                        1
School Code                          1
School Phone Number                  0
School Address                       0
School City                          0
School State                         0
School Zip                           1
School Not Found                     0
School or Citywide Complaint    364558
Vehicle Type                    364558
Taxi Company Borough            364558
Taxi Pick Up Location           364558
Bridge Highway Name             364261
Bridge Highway Direction        364261
Road Ramp                       364296
Bridge Highway Segment          364296
Garage Lot Name                 364558
Ferry Direction                 364557
Ferry Terminal Name             364556
Latitude                          4030
Longitude                         4030
Location                          4030
dtype: int64
```

Task 2: Perform basic data exploratory analysis:

2.1 Remove records with null Closed Date

```
import matplotlib.pyplot as plt

df.isnull().sum().plot(kind='bar', figsize=(12,5))
```

```
plt.title("Missing Values")
plt.show()
```



2.2 Remove Closed Date Null

```
df = df[df['Closed Date'].notnull()]
```

2.3 Date Analysis

```
df['Created Date'] = pd.to_datetime(df['Created Date'])
df['Closed Date'] = pd.to_datetime(df['Closed Date'])

df['Request_Closing_Time'] = df['Closed Date'] - df['Created Date']

df['Request_Closing_Time'] = df['Request_Closing_Time'].dt.total_seconds()
df = df[df['Closed Date'] >= df['Created Date']]
```

```
C:\Users\test\AppData\Local\Temp\ipykernel_3504\3835860286.py:1: UserWarning: Could not infer format, so each element will be pa
  df['Created Date'] = pd.to_datetime(df['Created Date'])
C:\Users\test\AppData\Local\Temp\ipykernel_3504\3835860286.py:2: UserWarning: Could not infer format, so each element will be pa
  df['Closed Date'] = pd.to_datetime(df['Closed Date'])
```

2.3.3 Descriptive Statistics

```
df['Request_Closing_Time'].describe()
```

```
count    3.621770e+05
mean     1.511330e+04
std      2.110255e+04
min      6.100000e+01
25%      4.533000e+03
50%      9.616000e+03
75%      1.887800e+04
```

```
max      2.134342e+06
Name: Request_Closing_Time, dtype: float64
```

2.3.4 Check Null

```
print(df[['Complaint Type','City']].isnull().sum())
```

```
Complaint Type      0
City              674
dtype: int64
```

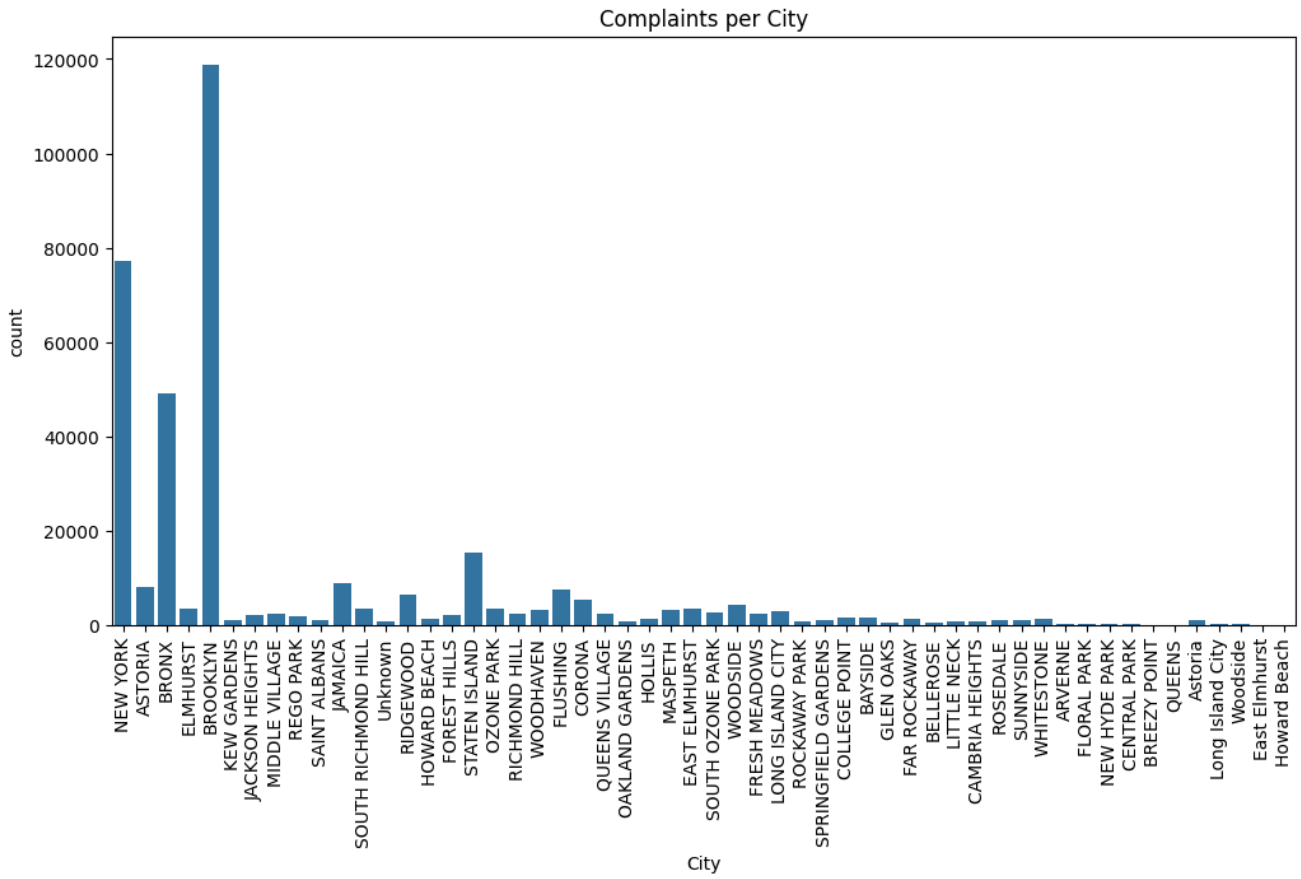
2.3.5 Fill Missing City

```
df['City'].fillna('Unknown', inplace=True)
```

```
C:\Users\test\AppData\Local\Temp\ipykernel_3504\1926131427.py:1: FutureWarning: A value is trying to be set on a copy of a DataF
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df


  df['City'].fillna('Unknown', inplace=True)
```

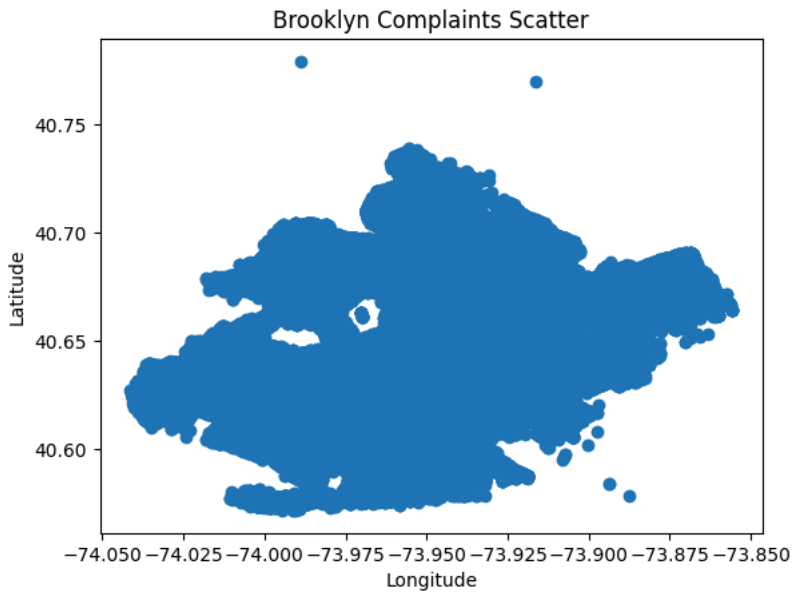
2.3.6 City Graph

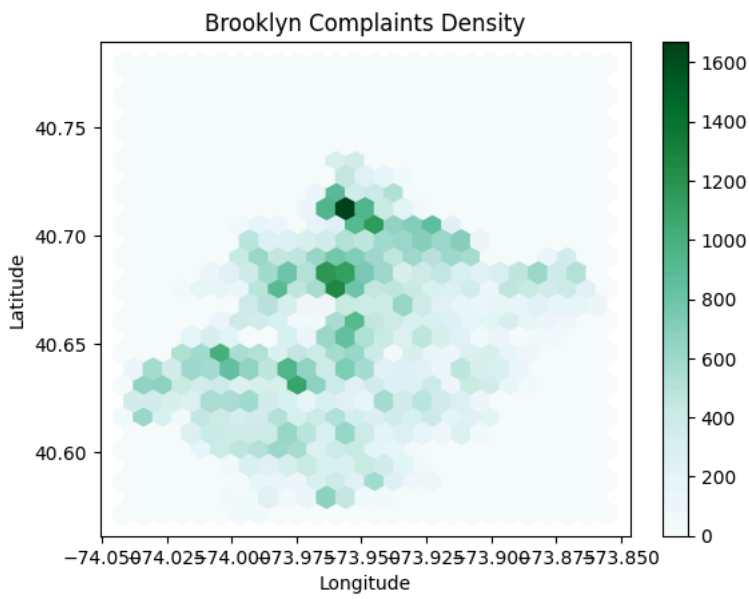
```
import seaborn as sns

plt.figure(figsize=(12,6))
sns.countplot(data=df, x='City')
plt.xticks(rotation=90)
plt.title("Complaints per City")
plt.show()
```



2.3.7 Brooklyn Graphs

```
brooklyn = df[df['City'] == 'BROOKLYN']

# Scatter Plot
plt.scatter(brooklyn['Longitude'], brooklyn['Latitude'])
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("Brooklyn Complaints Scatter")
plt.show()

# Hexbin Plot
brooklyn.plot.hexbin(x='Longitude', y='Latitude', gridsize=25)
plt.title("Brooklyn Complaints Density")
plt.show()
```

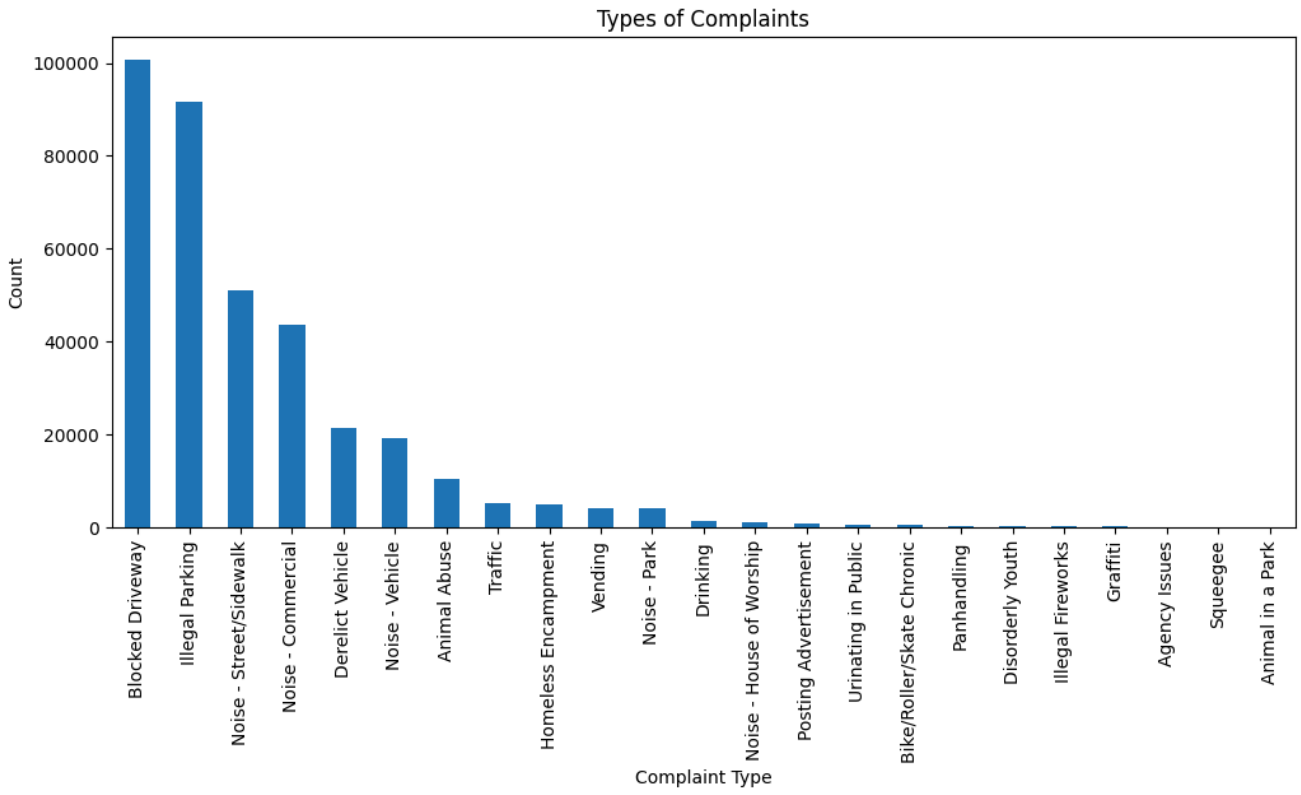




Task 3: Find major types of complaints:

3.1 Complaint Types Graph

```
df['Complaint Type'].value_counts().plot(kind='bar', figsize=(12,5))
plt.title("Types of Complaints")
plt.xlabel("Complaint Type")
plt.ylabel("Count")
plt.show()
```



3.2 NYC Complaints

```
nyc = df[df['City'] == 'NEW YORK']

nyc['Complaint Type'].value_counts()
```

```
Complaint Type
Noise - Street/Sidewalk      22245
Noise - Commercial           18686
Illegal Parking              14549
Noise - Vehicle               6294
Homeless Encampment           3060
Blocked Driveway              2705
Vending                       2638
Animal Abuse                  1941
Traffic                       1769
Noise - Park                  1243
Derelict Vehicle               695
Drinking                       321
Urinating in Public            264
Bike/Roller/Skate Chronic      254
Noise - House of Worship       222
Panhandling                    206
Disorderly Youth                81
Posting Advertisement           49
Illegal Fireworks               38
Graffiti                        25
Squeegee                         4
Name: count, dtype: int64
```

3.3 Top 10 Complaints

```
df['Complaint Type'].value_counts().head(10)
```

```
Complaint Type
Blocked Driveway           100624
Illegal Parking             91716
Noise - Street/Sidewalk     51139
Noise - Commercial          43751
```

```
Derelict Vehicle            21518
Noise - Vehicle             19301
Animal Abuse                10530
Traffic                      5196
Homeless Encampment          4879
Vending                      4185
Name: count, dtype: int64
```

3.4 & 3.5 Table

```
pd.crosstab(df['City'], df['Complaint Type'])
df_new = pd.crosstab(df['Complaint Type'], df['City'])

df_new.head()
```

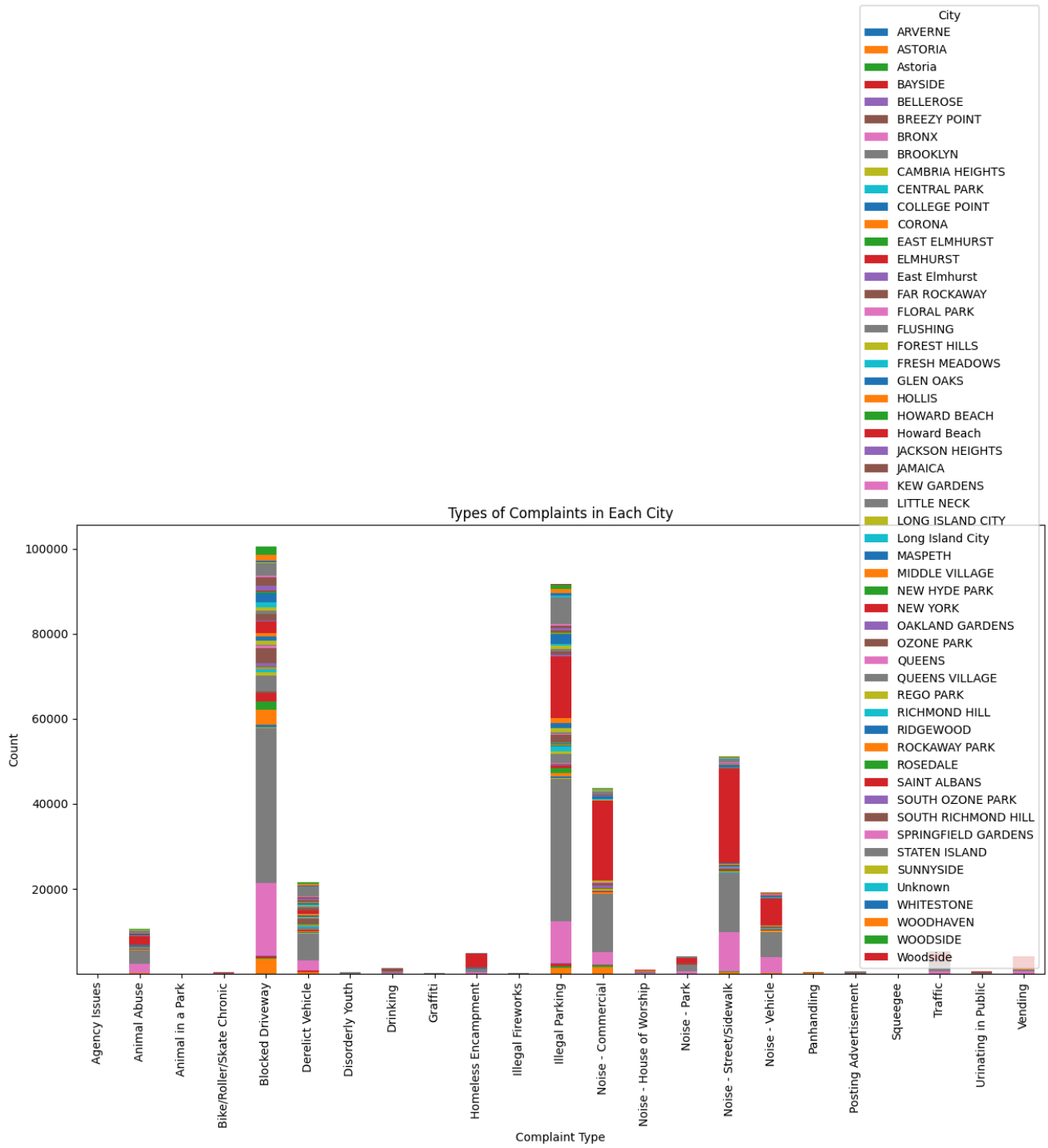
City	ARVERNE	ASTORIA	Astoria	BAYSIDE	BELLEROSE	BREEZY POINT	BRONX	BROOKLYN	CAMBRIA HEIGHTS	CENTRAL PARK	...	SOUTH OZONE PARK	SOUTH RICHMOND HILL	S
Complaint Type														
Agency Issues	0	0	0	0	0	0	0	0	0	0	...	0	0	
Animal Abuse	46	170	0	53	15	2	1971	3191	15	0	...	74	40	
Animal in a Park	0	0	0	0	0	0	0	0	0	0	...	0	0	
Bike/Roller/Skate Chronic	0	16	0	0	1	0	22	124	0	0	...	1	1	
Blocked Driveway	50	3436	159	514	138	3	17062	36445	177	0	...	1202	1946	

5 rows × 54 columns

Task 4: Visualize the major types of complaints in each city

4.1 Combined Graph

```
df_new.plot(kind='bar', stacked=True, figsize=(15,7))

plt.title("Types of Complaints in Each City")
plt.xlabel("Complaint Type")
plt.ylabel("Count")
plt.legend(title="City")
plt.show()
```



4.2 Response Time

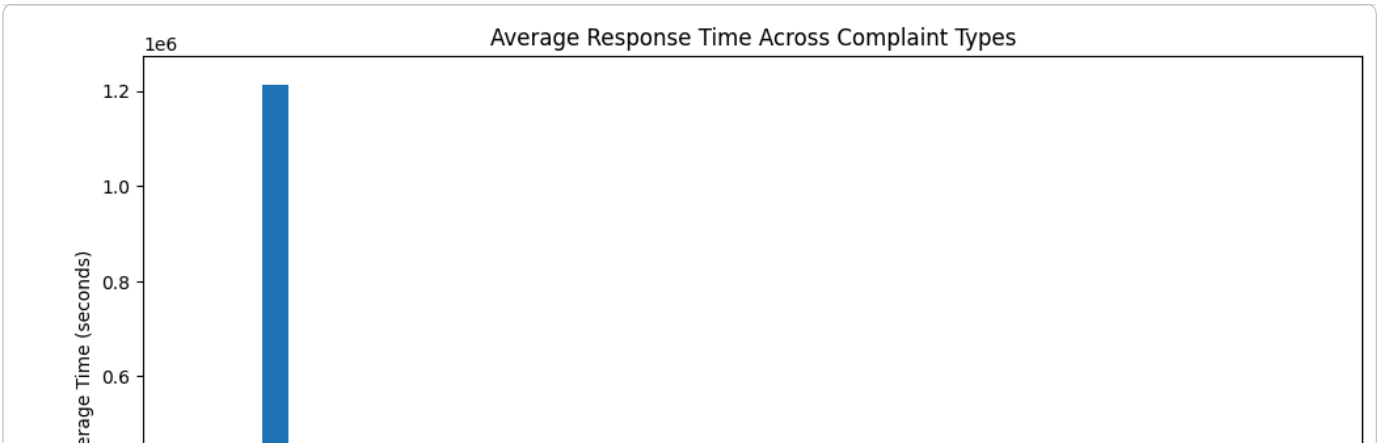
```
# Average closing time grouped by City and Complaint Type
avg_time_city = df.groupby(['City', 'Complaint Type'])['Request_Closing_Time'].mean()

# Sort values
```

```
avg_time_city = avg_time_city.sort_values()

# Convert to DataFrame for better display
avg_time_city = avg_time_city.reset_index()

# Show top 20 results
avg_time_city.head(20)
```

	City	Complaint Type	Request_Closing_Time
0	ARVERNE	Drinking	859.0
1	RIDGEWOOD	Posting Advertisement	895.0
2	EAST ELMHURST	Bike/Roller/Skate Chronic	908.0
3	OZONE PARK	Illegal Fireworks	1191.0
4	QUEENS	Urinating in Public	1226.0
5	SOUTH RICHMOND HILL	Illegal Fireworks	1712.5
6	ARVERNE	Vending	1735.0
7	SOUTH OZONE PARK	Illegal Fireworks	1821.0
8	KEW GARDENS	Posting Advertisement	1872.0
9	SUNNYSIDE	Graffiti	2133.0
10	COLLEGE POINT	Disorderly Youth	2160.0
11	FOREST HILLS	Illegal Fireworks	2427.0
12	FLUSHING	Illegal Fireworks	2476.5
13	ARVERNE	Urinating in Public	2491.0
14	JACKSON HEIGHTS	Illegal Fireworks	2520.0
15	ROCKAWAY PARK	Urinating in Public	2595.0
16	ELMHURST	Posting Advertisement	2640.0
17	SAINT ALBANS	Noise - Park	2953.0
18	ELMHURST	Disorderly Youth	3110.0
19	REGO PARK	Graffiti	3411.0

Task 5: See whether the average response time across different complaint types is similar (overall)

```
# Calculate average response time per complaint type
avg_time = df.groupby('Complaint Type')['Request_Closing_Time'].mean()

# Plot
avg_time.plot(kind='bar', figsize=(12,6))

plt.title("Average Response Time Across Complaint Types")
plt.xlabel("Complaint Type")
plt.ylabel("Average Time (seconds)")
plt.xticks(rotation=90)
plt.show()
```


Average Response Time Across Complaint Types
1e6
1.2
1.0
0.8
0.6
erage Time (seconds)